

บทที่ 3

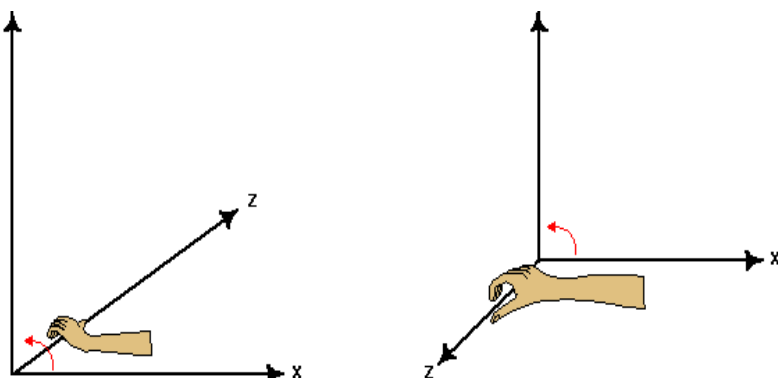
คณิตศาสตร์สำหรับการพัฒนาโปรแกรม 3 มิติ

3.1 ระบบพิกัดในปริภูมิ 3 มิติ (3D Coordinate Systems)

ระบบพิกัดที่ใช้ในโปรแกรม 3 มิติส่วนใหญ่เป็นระบบพิกัดคาร์ทีเซียน (Cartesian) ซึ่งมี 2 ระบบ คือ ระบบมือขวาและระบบมือซ้าย ระบบพิกัดทั้ง 2 ระบบนั้นแกน x ในส่วนที่มีค่าบวกจะชี้ไปทางขวามือ และแกน y ในส่วนที่มีค่าเป็นบวกก็จะชี้ขึ้นด้านบนเสมอ ส่วนทิศทางของแกน z สามารถหาได้โดยการใช้หลักมือซ้ายและมือขวาเช่นเดียวกับชื่อของระบบพิกัดแต่ละชนิด โดยถ้าเป็นระบบมือซ้ายก็ให้ใช้มือซ้ายและถ้าเป็นระบบมือขวาก็ให้ใช้มือขวา

วิธีหาทิศทางของแกน z

เริ่มต้นแบบมือออกแล้วชี้ปลายนิ้วมือไปในทิศทางของแกน x ด้านที่มีค่าเป็นบวก หลังจากนั้นให้งอนิ้วมือจนอยู่ในทิศทางของแกน y ด้านที่มีค่าเป็นบวก ทิศทางของหัวมือคือทิศทางของแกน z ในด้านที่มีค่าเป็นบวกซึ่งจะมีทิศทางชี้เข้าหรือออกจากตัวเราเสมอ



รูปที่ 3-1 ภาพแสดงระบบแกนในปริภูมิ 3 มิติ

3.2 จุดหรือเวกเตอร์ (Point or Vector)

จุดหรือเวกเตอร์เป็นข้อมูลที่มีความสำคัญมากในกราฟิก 3 มิติ เนื่องจากข้อมูลต่างๆ ไม่ว่าจะเป็นเวอร์เท็กซ์ (Vertex) ซึ่งใช้เป็นข้อมูลในแสดงโมเดล, ตำแหน่งและทิศทางของวัตถุ และความเร็ว/แรง ทั้งหมดสามารถแทนค่าในรูปของจุดในระนาบ 3 มิติหรือเวกเตอร์ได้ทั้งหมด ในปริภูมิ 3 มิติวัตถุประกอบด้วย ความกว้าง (width) คือ ค่าในแกน x, ความยาว (height) คือ ค่าในแกน y และความลึก (depth) คือ ค่าในแกน z

การแสดงค่าของจุดหรือเวกเตอร์

$$v = (x, y, z)$$

การแสดงค่าของจุดหรือเวกเตอร์ในรูปเมตริกซ์

$$v = \begin{bmatrix} x & y & z \end{bmatrix}$$

เนื่องจากคำว่าจุดและเวกเตอร์มีความหมายไปในทางเดียวกัน คือ

จุด หมายถึง ตำแหน่งในปริภูมิ 3 มิติ เวกเตอร์ หมายถึง เส้นที่ลากจากตำแหน่งอ้างอิง (จุดกำเนิด) มายังตำแหน่งในปริภูมิ 3 มิติ เวกเตอร์มีคุณสมบัติมากกว่าจุดคือ สามารถบอกทิศทางได้ ในที่นี้จึงใช้สลับกันได้ แต่ถ้าใช้ในการคำนวณจะกล่าวถึงในรูปของเวกเตอร์

การหาความยาวของเวกเตอร์

$$\|v\| = \sqrt{x^2 + y^2 + z^2}$$

การหาเวกเตอร์หนึ่งหน่วย (unit vector)

เวกเตอร์หนึ่ง คือ เวกเตอร์ที่มีขนาดหนึ่งหน่วยเสมอ ไม่ว่าจะอยู่ในทิศทางใดก็ตาม

$$u = \frac{v}{\|v\|}$$

มีเวกเตอร์หนึ่งหน่วย ที่ใช้แสดงทิศทางในแนวแกนต่างๆ ดังนี้

$$\text{แกน } x: i = (1, 0, 0)$$

$$\text{แกน } y: j = (0, 1, 0)$$

$$\text{แกน } z: k = (0, 0, 1)$$

เนื่องจากเวกเตอร์มีการนำไปใช้มากในทางฟิสิกส์ซึ่งมักจะเขียนเวกเตอร์ในรูปของค่าเวกเตอร์หนึ่งหน่วย i, j, k ทำให้เราสามารถแสดงค่าจุดในปริภูมิ 3 มิติในรูปของเวกเตอร์ได้โดยนำค่าในแต่ละแกนมาคูณกับเวกเตอร์หนึ่งหน่วยเหล่านี้ เช่น ถ้าต้องการแสดงจุด $a = (3, 5, 2)$ ในรูปของเวกเตอร์ จะได้

$$a = 3i + 5j + 2k$$

วิธีการแสดงค่าเวกเตอร์ในรูปเวกเตอร์หนึ่งหน่วยจะมีความสำคัญในการคำนวณค่ากับเมตริกซ์

ผลคูณเชิงสเกลาร์ของเวกเตอร์ (Scalar product or Dot product)

คือ ผลคูณของเวกเตอร์ 2 เวกเตอร์ที่ได้ผลลัพธ์เป็นสเกลาร์ ซึ่งค่าสเกลาร์ที่ได้แสดงค่าความยาวของเงาที่ฉายอยู่บนอีกเวกเตอร์หนึ่ง

ในระนาบ 2 มิติ

$$u \cdot v = u_x \times v_x + u_y \times v_y$$

$$u \cdot v = \|u\| \times \|v\| \times \cos(\theta)$$

ในระนาบ 3 มิติ

$$u \cdot v = u_x \times v_x + u_y \times v_y + u_z \times v_z$$

ผลคูณเชิงเวกเตอร์ของเวกเตอร์ (Cross Product)

คือ ผลคูณของเวกเตอร์ 2 เวกเตอร์ที่ได้ผลลัพธ์เป็นเวกเตอร์ ซึ่งเวกเตอร์ที่ได้จะตั้งฉากกับเวกเตอร์ทั้งสองเสมอ

$$u \times v = (u_y v_z - u_z v_y, u_z v_x - u_x v_z, u_x v_y - u_y v_x)$$

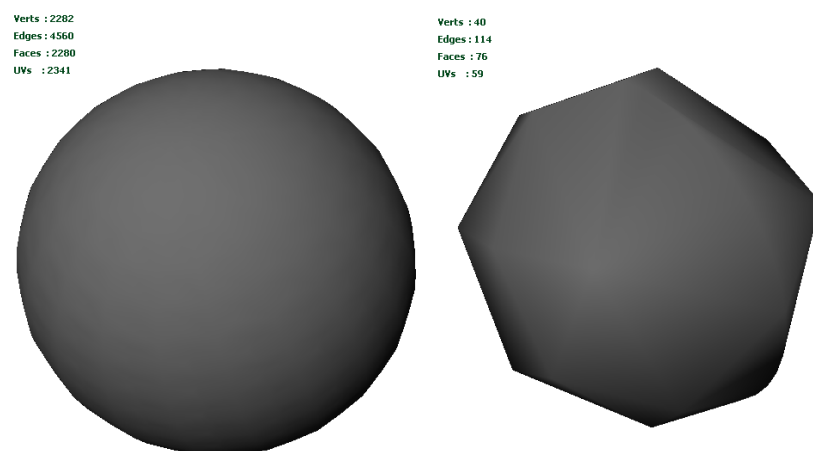
การคูณเชิงเวกเตอร์ของ 2 เวกเตอร์ไม่มีคุณสมบัติในการสลับที่ ดังนั้น $u \times v$ ไม่เท่ากับ $v \times u$ และถ้า 2 เวกเตอร์ขนานกันจะหาค่าผลคูณไม่ได้ (อาจจะได้เวกเตอร์ที่มีค่าอนันต์ (infinity) ซึ่งตั้งฉากกับเวกเตอร์หนึ่ง)

3.3 โพลีกอน (Polygon)

โพลีกอนเป็นพื้นฐานในสาขาวิชาคอมพิวเตอร์กราฟิก ถึงแม้ว่าในปัจจุบันจะมีเทคโนโลยีอย่างอื่น เช่น NURBS เป็นต้น แต่อุปกรณ์ที่ใช้กับเครื่องคอมพิวเตอร์ส่วนบุคคลทุกวันนี้ส่วนใหญ่ยังคงใช้หลักการวาดรูปด้วยโพลีกอน

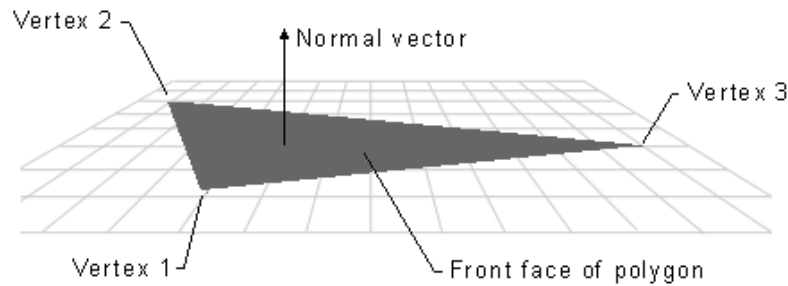
โคเร็กทรีตีใช้โพลีกอนในการสร้างภาพของวัตถุ ซึ่งโพลีกอน ก็คือ พื้นผิวที่เกิดจากเส้นรอบรูปที่มีลักษณะเป็นวงปิด (โพลีกอนสนใจเฉพาะพื้นผิวไม่สนใจปริมาตรของวัตถุ) ซึ่งเส้นรอบรูปของโพลีกอนจะมีรูปร่างเป็นรูปทรงเรขาคณิต เช่น สามเหลี่ยม, สี่เหลี่ยม เป็นต้น หลักการในการสร้างรูปทรงของวัตถุโดยใช้โพลีกอนก็คือ การใช้พื้นผิวพื้นฐานที่สุด (primitive) ประกอบกันในการสร้างรูปทรงที่มีความซับซ้อนมากๆ ขึ้นไป และรูปทรงที่พื้นฐานที่สุดก็คือ รูปสามเหลี่ยม นั่นเอง ดังนั้นโดยอาศัยหลักการของโพลีกอนเราสามารถสร้างพื้นผิวของวัตถุที่มีรูปร่างใดๆ ก็ได้โดยไม่มีข้อจำกัด

อย่างไรก็ตาม รูปทรงที่สร้างขึ้นจากโพลีกอนจะเป็นการประมาณค่าพื้นผิวของวัตถุโดยใช้รูปสามเหลี่ยมกลุ่มค่าจุดทั้งสามชิ้นมาจากพื้นผิวจริง ซึ่งถ้าจำนวนครั้งที่เรากลุ่มน้อยเกินไป (มีรูปสามเหลี่ยมน้อย) ก็อาจจะทำให้วัตถุที่มีความซับซ้อนสูงๆ มีการคาดเคลื่อนไป เช่น ถ้าเราใช้รูปสามเหลี่ยมจำนวนน้อยในการสร้างรูปทรงกลมจะทำให้ได้รูปหลายเหลี่ยมแทน



รูปที่ 3-2 รูปแสดงความแตกต่างระหว่างจำนวนของรูปสามเหลี่ยมที่ใช้ในการกลุ่มค่าจากวัตถุทรงกลม

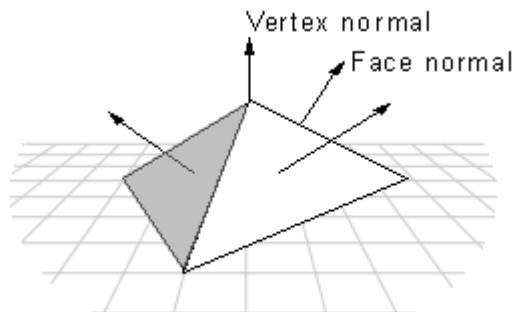
โครงสร้างข้อมูลของ polygon



รูปที่ 3-3 โครงสร้างข้อมูลที่ประกอบขึ้นมาเป็นพื้นผิว

ในการสร้างรูปสามเหลี่ยม 1 รูป จะต้องใช้จุดสามจุดในการกำหนดตำแหน่งความยาวของด้านแต่ละด้านของโพลีกอน ซึ่งจุดบนโพลีกอนมีชื่อเรียกว่า เวอร์เท็กซ์เนื่องจากมันไม่ได้เก็บเฉพาะข้อมูลเกี่ยวกับตำแหน่งเท่านั้น แต่ยังประกอบด้วยข้อมูลอื่นๆ ที่ใช้กำหนดคุณลักษณะของพื้นผิวที่จะปรากฏอีกด้วย

ข้อมูลเหล่านั้นใดเรักรหัสสามารถนำไปประมวลผลเพื่อสร้างภาพที่มีความสมจริงยิ่งขึ้นได้ เช่น ค่าเวกเตอร์ตั้งฉากของเวอร์เท็กซ์ (normal vector) สามารถนำไปใช้คำนวณมุมที่แสงกระทำต่อวัตถุทำให้ได้วัตถุที่มีสีส้มเหมือนจริงยิ่งขึ้น, ข้อมูลน้ำหนักในการเบลนด์ (blend weight data) ใช้ในการทำแอนิเมชันให้กับวัตถุ ซึ่งทำให้ได้วัตถุที่มีการเคลื่อนไหวอย่างเป็นธรรมชาติ และคู่อันดับของเทกเจอร์ (texture coordinate) ใช้กำหนดคู่อันดับของภาพที่จะปะบนวัตถุทำให้วัตถุที่มีสีส้มของพื้นผิวเหมือนจริงและเป็นธรรมชาติมากขึ้น เป็นต้น



รูปที่ 3-4 เวกเตอร์ที่ตั้งฉากกับเวอร์เท็กซ์และเวกเตอร์ที่ตั้งฉากกับพื้นผิว

3.4 ระนาบ (Plane)

เป็นข้อมูลพื้นฐานที่สำคัญอีกอย่างหนึ่ง ระนาบในทาง 3 มิติก็มีความหมายเดียวกับเส้นตรงที่มีความยาวอนันต์ใน 2 มิติ หรือก็คือ สิ่งที่อยู่ในมิติ $n-1$ นั่นเอง ซึ่งจะแบ่งปริภูมิหรือระนาบเป็น 2 ส่วนแยกจากกันโดยเด็ดขาด สามารถนำไปงานใช้ในการคำนวณหลายๆ อย่างใน 3 มิติ เช่น การหาว่าจุดหรือโพลีกอนอยู่ในระนาบเดียวกันหรือไม่, ใช้ในการแบ่งโพลีกอนที่อยู่ในบริเวณที่จะแสดงผล (clipping) และใช้ในการคำนวณหา BSP Tree เป็นต้น

สมการระนาบในปริภูมิ 3 มิติ คือ

$$ax + by + cz + d = 0$$

นอกจากนั้น เรายังสามารถหาค่าอื่นๆ ที่เกี่ยวข้องกับระนาบได้ ดังนี้

1. ค่า (a, b, c) คือเวกเตอร์ที่ตั้งฉากกับระนาบ
2. ค่า d คือระยะทางตั้งฉากจากระนาบกับจุดกำเนิด
3. เมื่อนำค่าจุดใดๆ มาแทนค่าใน x, y, z สามารถหาว่าจุดนั้นอยู่ในระนาบหรือไม่ ถ้าอยู่ในระนาบจะได้ค่าผลลัพธ์เป็น 0

3.5 เมตริกซ์ (Matrix)

เมตริกซ์ช่วยให้เราสามารถแสดงสมการหลายๆ สมการได้พร้อมๆ กัน เช่น ถ้าเราต้องการจะแก้สมการที่มีตัวแปร x, y, z ดังนี้

$$3x - 8y + 12z = 0$$

$$15x + 14y - 2z = 0$$

$$32x + 0.5y - z = 0$$

เมื่อนำค่าสัมประสิทธิ์ของสมการมาใส่ในกรอบขนาด n แถว m ที่เรียกว่า เมตริกซ์ เมื่อ n คือจำนวนของคอลัมน์ (vertical dimension) และ m คือ จำนวนของแถว (horizontal dimension) จะได้

$$\begin{aligned} 3x - 8y + 12z &= 0 \\ 15x + 14y - 2z &= 0 \\ 32x + 0.5y - z &= 0 \end{aligned} \Rightarrow \begin{bmatrix} 3 & -8 & 12 \\ 15 & 14 & -2 \\ 32 & 0.5 & 1 \end{bmatrix}$$

สัญลักษณ์ที่ใช้แทนเมตริกซ์ขนาด $m \times n$

$$A_{m \times n} = [a_{ij}]_{m \times n} = \begin{bmatrix} a_{11} & a_{12} & \cdot & \cdot & \cdot & a_{1n} \\ a_{21} & a_{22} & \cdot & \cdot & \cdot & a_{2n} \\ \cdot & \cdot & & & & \cdot \\ \cdot & \cdot & & & & \cdot \\ \cdot & \cdot & & & & \cdot \\ a_{m1} & a_{m2} & \cdot & \cdot & \cdot & a_{mn} \end{bmatrix}$$

การบวกเมตริกซ์

นิยาม

ถ้า $A = [a_{ij}]_{m \times n}$ และ $B = [b_{ij}]_{m \times n}$ แล้ว $A + B = [a_{ij} + b_{ij}]_{m \times n}$

ตัวอย่าง เช่น

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} + \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{bmatrix}$$

การคูณเมตริกซ์

นิยาม

ถ้า $A = [a_{ij}]_{m \times n}$ และ $B = [b_{ij}]_{n \times r}$ แล้ว ผลคูณ $A \times B$ คือ เมตริกซ์ $C = [c_{ij}]_{m \times r}$ โดยที่

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{in}b_{nj}$$

ตัวอย่าง เช่น

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} + \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}$$

การคูณเมตริกซ์ไม่มีคุณสมบัติในการสลับที่ คือ $A \times B \neq B \times A$ ดังนั้น ต้องระวังในการคูณเมตริกซ์

ตัวอย่าง การคูณค่าของจุดกับเมตริกซ์

$$vA = v'$$

$$\begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} x' & y' & z' \end{bmatrix}$$

เมตริกซ์มีการนำไปใช้ในสาขาคอมพิวเตอร์กราฟิกอย่างมาก เนื่องจากการใช้เมตริกซ์ทำให้เราสามารถทำการย้ายตำแหน่งของจุด, หมุน หรือย่อขยายได้อย่างรวดเร็วด้วยการนำจุดมาคูณหรือบวกกับเมตริกซ์เท่านั้น แต่การใช้เมตริกซ์ 3x3 จะทำให้ได้ค่าที่ไม่ถูกต้อง เนื่องจากความไม่เข้ากันของการคูณและการบวกในเมตริกซ์ ดังนั้นเพื่อแก้ปัญหานี้จึงต้องเพิ่มองค์ประกอบในเมตริกซ์ให้เป็นเมตริกซ์ขนาด 4x4 โดยองค์ประกอบที่เพิ่มขึ้นมา เรียกว่า homogenous coordinate ซึ่งแทนด้วยตัวอักษร w

ตัวอย่างการเพิ่ม homogenous coordinate ให้กับค่าจุดในระบบคู่อันดับคาร์ทีเซียน

$$\begin{bmatrix} x & y & z \end{bmatrix} \Rightarrow \begin{bmatrix} bx & by & bz & b \end{bmatrix}$$

โดย b จะเป็นค่าใดๆ ก็ได้ที่ไม่เท่ากับ 0 (ปกติจะใช้ b = 1)

การย้ายตำแหน่งจุด (translation)

ให้ $p = [p_x \ p_y \ p_z]$ เป็นเวกเตอร์แทนระยะทางที่ต้องการย้ายจุดไป จะได้เมตริกซ์สำหรับการย้ายจุด (translation matrix) ดังนี้

$$T(p) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ p_x & p_y & p_z & 1 \end{bmatrix}$$

นั่นคือ จะต้องนำจุดที่ต้องการจะย้ายตำแหน่งมาคูณกับเมตริกซ์สำหรับการย้ายจุด ดังนี้

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ p_x & p_y & p_z & 1 \end{bmatrix} = \begin{bmatrix} x + p_x & y + p_y & z + p_z & 1 \end{bmatrix}$$

การหมุนตำแหน่งจุดรอบแกน (rotation)

การหมุนจะต้องทำโดยอ้างอิงกันแกนหมุน ซึ่งก็คือ แกน x, แกน y และแกน z

เมตริกซ์ที่ใช้ในการหมุน (rotation matrix) รอบแกน x

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & \sin(\theta) & 0 \\ 0 & -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

เมตริกซ์ที่ใช้ในการหมุน (rotation matrix) รอบแกน y

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & -\sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ \sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

เมตริกซ์ที่ใช้ในการหมุน (rotation matrix) รอบแกน z

$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 & 0 \\ -\sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

โดย ค่า θ คือ มุมที่ต้องการหมุนจากแกนนั้นๆ

การย่อขยายกลุ่มของจุด (scale)

$$S = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

นอกจากการใช้เมตริกซ์เพื่อการย้ายตำแหน่งของจุด, การหมุนตำแหน่งของจุดและการย่อขยายตำแหน่งของจุด แล้วยังสามารถใช้ข้อมูลในเมตริกซ์ในการหาค่าอื่นๆ ได้อีก ดังนี้

เวกเตอร์ p แสดงความสัมพันธ์ของวัตถุกับจุดกำเนิด

เวกเตอร์ n, o และ a แสดงทิศทางในแต่ละแกนหรือในรูปเวกเตอร์หนึ่งหน่วย i, j และ k

ซึ่งค่าเวกเตอร์เหล่านี้สามารถหาได้จากเมตริกซ์ในตำแหน่งต่างๆ ดังนี้

$$\begin{bmatrix} n_x & n_y & n_z & 0 \\ o_x & o_y & o_z & 0 \\ a_x & a_y & a_z & 0 \\ p_x & p_y & p_z & 1 \end{bmatrix}$$

3.6 ควอเทอร์เนียน (Quaternion)

ควอเทอร์เนียนช่วยอำนวยความสะดวกในกระบวนการหมุนตำแหน่งของจุด และมีความเร็วกว่าการคำนวณโดยใช้เมตริกซ์ ประกอบด้วยองค์ประกอบ 2 ส่วน คือ ค่าเวกเตอร์และค่าสเกลาร์ ควอเทอร์เนียนสามารถเขียนในรูปแบบต่างๆ ได้ ดังนี้

$$\begin{aligned} q &= (x, y, z, w) \\ q &= xi + yj + zk + w \\ q &= [x \ y \ z \ w] \\ q &= [sv] \end{aligned}$$

เมื่อ x, y, z เป็นค่าสัมประสิทธิ์ของเวกเตอร์หนึ่งหน่วย i, j, k
 w เป็นค่าสเกลาร์
 $s = w$ และ $v = [x \ y \ z]$

การคูณควอเทอร์เนียน (quaternion multiplication)

เนื่องจากควอเทอร์เนียนที่เราใช้เป็นสับเซตของควอเทอร์เนียนทั่วไป (General quaternion) ซึ่งเป็นการขยายความสามารถของจำนวนเชิงซ้อน (extension of complex number) ดังนั้น การคูณควอเทอร์เนียน จะต้องนำคุณสมบัติการคูณกันของจำนวนเชิงซ้อนมาช่วย ดังนี้

$$\begin{aligned} i \times i &= -1 \\ j \times j &= -1 \\ k \times k &= -1 \\ i \times j &= -j \times i = k \\ j \times k &= -k \times j = i \\ k \times i &= -i \times k = j \end{aligned}$$

เนื่องจากในระบบจำนวนจริง (real number) ค่า i, j และ k จะเป็นแกนในแนวต่างๆ นั้นเอง ดังนั้น การคูณควอเทอร์เนียนจึงอยู่ในรูปของการคูณจำนวนเชิงซ้อน ดังนี้

$$\begin{aligned} q_1 q_2 &= (w_1 + x_1 i + y_1 j + z_1 k)(w_2 + x_2 + y_2 + z_2) \\ &= (w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2) + (y_1 z_2 - y_1 z_2 + w_1 x_2 + w_1 x_2) i + \\ &\quad (z_1 x_2 - z_1 x_2 + w_1 y_2 + w_1 y_2) j + (x_1 y_2 - x_1 y_2 + w_1 z_2 + w_1 z_2) k \end{aligned}$$

การหมุนตำแหน่งของจุดโดยใช้ควอเทอร์เนียน

การหมุนเวกเตอร์ $P = [x \ y \ z]$ ทำได้โดยการใช้ควอเทอร์เนียนบริสุทธิ์ (pure quaternion) p และคอนจูเกต (conjugate) ของ q

ควอเตอร์เนียนบริสุทธิ์ของ p

$$p = xi + yj + zk$$

ค่าของควอเตอร์เนียนที่ใช้ในการหมุนรอบแกน $A(x, y, z)$ ด้วยมุม θ คือ

$$q = (sx, sy, sz, c)$$

เมื่อ

$$s = \sin(\theta/2)$$

$$c = \cos(\theta/2)$$

คอนจูเกตของควอเตอร์เนียน

$$q^* = [-x \ -y \ -z \ w]$$

การหมุนตำแหน่งจุดโดยใช้ควอเตอร์เนียน

$$Rot_q(P) = qpq^*$$